Assignment 10

Name: Kabir Patil

Roll No: B-26

Subject: Agentic AI

Task 20: Multi-Agent Research and Report Generation

Objective:

Create a multi-agent system where a Research Agent gathers key information on a topic and a Writer Agent converts the research notes into a structured report.

Code:

```
import os
from dotenv import load_dotenv
from huggingface_hub import InferenceClient

load_dotenv()
token = os.getenv("HF_TOKEN")

client = InferenceClient(
    api_key=token,
    provider="featherless-ai"
)

topic = input("Research topic: ")

# Research Agent
research_prompt = f"""
You are a research agent.

Research important points about:
{topic}

Give concise and useful research notes.
"""

research_response = client.chat.completions.create(
    model="meta-llama/Llama-3.2-1B-Instruct",
    messages=[
        {"role": "user", "content": research_prompt}
    ],
    max_tokens=120
)
```

```
research = research_response.choices[0].message.content

# Writer Agent
writer_prompt = f"""
You are a report writer.

Convert these research notes into
a short structured report:

{research}
"""

summary_response = client.chat.completions.create(
    model="meta-llama/Llama-3.2-1B-Instruct",
    messages=[
        {"role": "user", "content": writer_prompt}
    ],
    max_tokens=250
)

summary = summary_response.choices[0].message.content

print("\nResearch Agent:")
print(research)

print("\nWriter Agent:")
print(summary)
```

Output / Result:

```
Research topic: AI-assisted Development

Research Agent:
**Research Notes: AI-assisted Development**

**Overview**

Artificial intelligence (AI) is transforming various industries, including development,
by providing innovative solutions and enhancing the efficiency of processes. AI-assisted
development leverages AI technologies to improve the design, testing, and deployment of
products and services.

**Key Benefits**

1. **Improved Accuracy**: AI algorithms can analyze vast amounts of data, reducing the
likelihood of human error and improving the accuracy of designs.
2. **Faster Time-to-Market**: AI-assisted development enables faster prototyping and
testing, allowing companies to bring products to market more

Writer Agent:
**Report: AI-assisted Development**

**Executive Summary**
```

Artificial intelligence (AI) is revolutionizing the development industry by providing innovative solutions and enhancing efficiency. AI-assisted development leverages AI technologies to improve product design, testing, and deployment. This report highlights the key benefits of AI-assisted development, including improved accuracy and faster time-to-market.

Introduction

Artificial intelligence (AI) has been transforming various industries, including development, by providing innovative solutions and enhancing efficiency. AI-assisted development is a critical aspect of this transformation, as it enables companies to improve product design, testing, and deployment.

Key Benefits of AI-assisted Development

1. **Improved Accuracy**: AI algorithms can analyze vast amounts of data, reducing the likelihood of human error and improving the accuracy of designs.
2. **Faster Time-to-Market**: AI-assisted development enables faster prototyping and testing, allowing companies to bring products to market more quickly.

Case Study: Example of AI-assisted Development

A leading software company used AI-assisted development to improve its product design process. By leveraging AI algorithms, the company was able to reduce design time by 30% and increase the accuracy of designs by 25%.

Task 21: Supervisor Routing and Specialist Agent Selection

Objective:

Create a supervisor-agent architecture that selects the appropriate specialist agent (MATH, PYTHON, WRITING) for a task and routes the request to solve it.

Code:

```
import os
from dotenv import load_dotenv
from huggingface_hub import InferenceClient

load_dotenv()
token = os.getenv("HF_TOKEN")

client = InferenceClient(
    api_key=token,
    provider="featherless-ai"
)

task = input("Enter task: ")

# Supervisor Agent
supervisor_prompt = f"""
You are a supervisor agent.

Choose the best specialist for the following task:

MATH
PYTHON
WRITING

Task:
{task}

Return only one specialist name:
MATH, PYTHON, or WRITING.
"""

supervisor_response = client.chat.completions.create(
    model="meta-llama/Llama-3.2-1B-Instruct",
    messages=[
        {"role": "user", "content": supervisor_prompt}
    ],
    max_tokens=20
)

expert = supervisor_response.choices[0].message.content.strip()

print("\nSupervisor selected:", expert)

# Specialized Agent
```

```
specialist_prompt = f"""
You are a {expert} specialist.

Solve the following task:

{task}

Give a clear and concise answer.
"""


answer_response = client.chat.completions.create(
    model="meta-llama/Llama-3.2-1B-Instruct",
    messages=[
        {"role": "user", "content": specialist_prompt}
    ],
    max_tokens=250
)

answer = answer_response.choices[0].message.content

print("\nSpecialist Agent:")
print(answer)
```

Output / Result:

```
Enter task: Write a simple java  code for adding two numbers

Supervisor selected: Based on the task, I would recommend the specialist "WRITING" for
this task. Here

Specialist Agent:
Here's a simple Java code for adding two numbers:

```java
public class Main {
    public static void main(String[] args) {
        // Declare variables for two numbers
        int num1 = 10;
        int num2 = 20;

        // Add the two numbers
        int sum = num1 + num2;

        // Print the sum
        System.out.println("The sum of " + num1 + " and " + num2 + " is " + sum);
    }
}
```
```

Task 22: API Weather Integration and LLM Advice Agent

Objective:

Create an agent that fetches real-time weather data for a city using an external API and uses an LLM to generate concise, practical user advice.

Code:

```
import os
import requests
from dotenv import load_dotenv
from huggingface_hub import InferenceClient

load_dotenv()
token = os.getenv("HF_TOKEN")

client = InferenceClient(
    api_key=token,
    provider="featherless-ai"
)

city = input("Enter city: ")

# Get weather data from API
url = f"https://wttr.in/{city}?format=j1"
data = requests.get(url).json()

temp = data["current_condition"][0]["temp_C"]
weather = data["current_condition"][0]["weatherDesc"][0]["value"]

# LLM Agent
prompt = f"""
You are a weather advice agent.

Give simple advice for someone in {city}.

Weather: {weather}
Temperature: {temp} C

Keep the advice short and practical.
"""

response = client.chat.completions.create(
    model="meta-llama/Llama-3.2-1B-Instruct",
    messages=[
        {"role": "user", "content": prompt}
    ],
    max_tokens=100
)

print("\nWeather:")
print(weather)
```

```
print("Temperature:", temp, "C")

print("\nAgent Advice:")
print(response.choices[0].message.content)
```

Output / Result:

```
Enter city: Pune

Weather:
Light rain shower
Temperature: 20 C

Agent Advice:
Given the light rain shower and 20°C temperature in Pune, here's some simple advice:

* Stay indoors and avoid traveling unless absolutely necessary.
* Keep your vehicle doors and windows closed to prevent rainwater from entering.
* If you must go out, wear waterproof gear and an umbrella to keep you dry.
```

Task 23: Computer Vision Classification and Decision Threshold Agent

Objective:

Create an agent that classifies images using a pre-trained Vision Transformer model and makes an automated decision based on confidence thresholding.

Code:

```
from pathlib import Path
from PIL import Image
from transformers import pipeline

classifier = pipeline(
    "image-classification",
    model="google/vit-base-patch16-224"
)

image_path = Path(input("Enter image path: ").strip().strip('"').strip("'"))

if not image_path.is_file():
    raise FileNotFoundError(f"Image not found: {image_path.resolve()}")

with Image.open(image_path) as image:
    results = classifier(image.convert("RGB"))

label = results[0]["label"]
score = results[0]["score"]

print("\nPrediction:", label)
print("Confidence:", round(score, 2))

if score > 0.7:
    print("Agent Action: Accept prediction")
else:
    print("Agent Action: Request human review")
```

Output / Result:

```
Enter image path: D:\Walpapers\porsche.jpeg

Prediction: sports car, sport car
Confidence: 0.96
Agent Action: Accept prediction
```